

L12D: A Metacircular Evaluator

CS1101S: Programming Methodology

Martin Henz

November 8, 2023

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation

On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation

On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

The most beautiful program ever written

Gerald Jay Sussman, CS1101S Guest Lecture 2020/21, 25:35

“The most beautiful program ever written, in my opinion, is the Eval/Apply Interpreter.”

Why is this beautiful?

It is beautiful in the same way as Maxwell's Equations on electromagnetism are beautiful: It conveys deep insights in a few lines.

On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

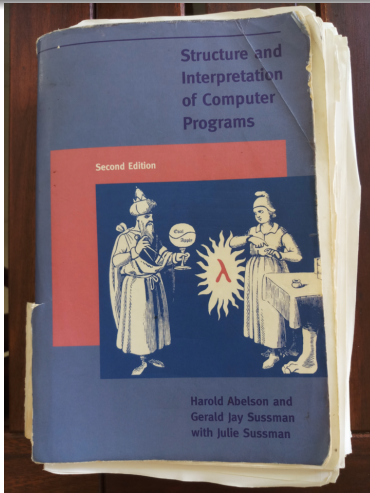
A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

SICP



On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

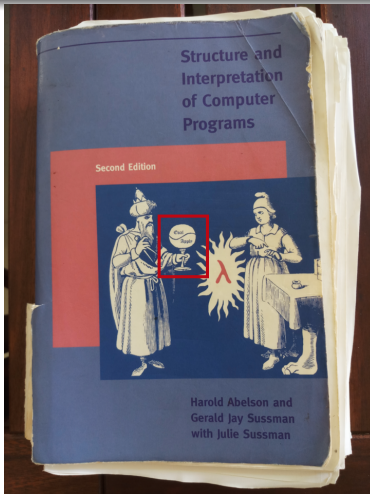
A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

SICP: What is that in the middle?



On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

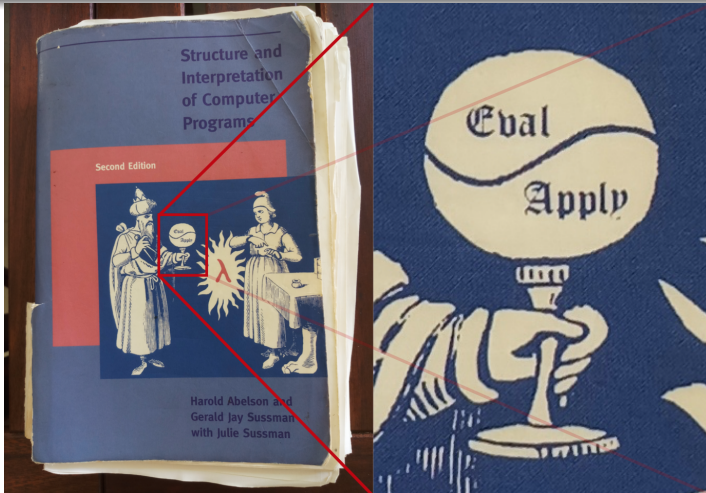
A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

SICP: The importance of Eval/Apply



On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

Metacircular evaluator vs CSE machine

CSE machine in Source

implements the CSE machine of Unit 3

Metacircular evaluator

follows Section 4.1 of SICP JS, using a recursive evaluator

This lecture

covers the metacircular evaluator of SICP JS 4.1 and lazy evaluation of SICP JS 4.2

On metacircular evaluators

Calculator language

Adding booleans, conditionals, and sequences

Adding blocks and declarations

Adding simple compound functions

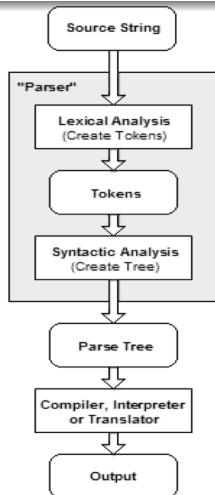
A small complication: return statements

Lazy Evaluation

Why a metacircular evaluator?

Metacircular evaluator vs CSE machine

Review: parsing



- Lexical analysis

- Input characters are split into meaningful symbols (tokens) defined using *regular expressions*
- Example: "8 + 34;" is split into tokens 8 + 34 ;

- Parsing

- Checks that the tokens form allowable components
- Produces symbolic representation of the program, a *syntax tree*

- 1 On metacircular evaluators
- 2 **Calculator language**
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation

Review: Calculator language in Backus-Naur Form (BNF)

stmt ::= *expr* ; expression statement

expr ::= *expr bin-op expr* binary operator combination

 | *number* number expression

 | (*expr*) parenthesised expression

bin-op ::= + | - | * | / binary operator

Example:

1.4 + 2.3 / 70.4;

Review: Parsing the calculator language

To run the program

```
1.4 + 2.3 / 70.4;
```

we first parse it:

```
parse("1.4 + 2.3 / 70.4;");
```

returns the syntax tree

```
list("binary_operator_comb",  
    "+",  
    list("literal", 1.4),  
    list("binary_operator_comb", "/",  
        list("literal", 2.3),  
        list("literal", 70.4)))
```

Review: Syntax of literals

Example: `parse("...38...");` $\Rightarrow$

`...list("literal", 38)...`

// syntax predicate (in the following in maroon)

```
function is_literal(comp) {
```

```
    return is_tagged_list(comp, "literal");
```

```
}
```

```
function is_tagged_list(comp, the_tag) {
```

```
    return is_pair(comp) && head(comp) === the_tag;
```

```
}
```

// selector (in the following in blue)

```
function literal_value(comp) {
```

```
    return head(tail(comp));
```

```
}
```

Review: Syntax of operator combinations

```
parse("... 2.3 + 70.4 ..."); ⇒
```

```
... list("binary_operator_comb", "+",  
        list("literal", 2.3),  
        list("literal", 70.4)) ...
```

```
// syntax predicate (parsing details in 4.1.2)
```

```
const is_operator_combination = comp =>
```

```
is_tagged_list(comp, "binary_operator_comb");
```

```
// selectors
```

```
const op_comb_symbol      = comp => list_ref(comp, 1);
```

```
const op_first_operand    = comp => list_ref(comp, 2);
```

```
const op_second_operand   = comp => list_ref(comp, 3);
```

Evaluating the calculator language

```
function evaluate(expr) {  
  return is_literal(expr)  
    ? literal_value(expr)  
    : is_operator_combination(expr)  
    ? apply(op_comb_operator_symbol(expr),  
            list_of_values(  
              list(op_comb_first_operand(expr),  
                  op_comb_second_operand(expr)))  
            : error(expr, "Unknown expression:");  
}
```



```
function list_of_values(exprs) {  
  return map(evaluate, exprs);  
}
```

Applying operator combinations

Example: 2.3 / 70.4

```
function apply(operator, operands)      {  
  const first_op = head(operands);  
  const second_op = head(tail(operands));  
  return operator === "+"  
    ? first_op + second_op  
    : operator === "-"  
    ? first_op - second_op  
    : operator === "*"  
    ? first_op * second_op  
    : operator === "/"  
    ? first_op / second_op : error(); }  
}
```


On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Conditional expressions
Sequences

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation

Review: Adding booleans, conditionals, and sequences

$stmt ::= \dots$
 | $stmt_1 \dots stmt_n$ statement sequence

 $expr ::= \dots$
 | true | false boolean literals
 | $expr_1 ? expr_2 : expr_3$ conditional expression

Example:

8 + 34; true ? 1 + 2 : 17;

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Conditional expressions
Sequences

Evaluating the language

```
function evaluate(comp) {  
  return ... // handling literals, operator comb  
    : is_conditional(comp)  
    ? eval_conditional(comp)  
    : is_sequence(comp)  
    ? eval_sequence(sequence_statements(comp))  
    : error(comp, "Unknown component:");  
}
```

Review: Parsing conditionals: an example

```
parse("true ? 42 : 17;");
```

returns

```
list("conditional_expression",  
    list("literal", true),  
    list("literal", 42),  
    list("literal", 17))
```

Syntax predicate: `is_conditional`

Selectors: `cond_expr_pred`, `cond_expr_cons`, `cond_expr_alt`

Evaluating conditionals

Example: `true ? 42 : 17;`

```
function eval_conditional(comp) {  
  return is_truthy(  
    evaluate(cond_expr_pred(comp))  
    ? evaluate(cond_expr_cons(comp))  
    : evaluate(cond_expr_alt(comp));  
}  
  
const is_truthy = x =>  
  is_boolean(x) ? x : error();
```

Here, we explain conditionals in the interpreted language using conditional expressions in the interpreting language.

Review: Parsing sequences: an example

```
parse("1; 2; 3;");
```

returns

```
list("sequence",  
     list(list("literal", 1),  
          list("literal", 2),  
          list("literal", 3))
```

Syntax predicate: `is_sequence`

Selectors: `sequence_statements`, `first_statement`,
`rest_statements`

More syntax predicates: `is_empty_sequence`,
`is_last_statement`

Evaluating sequences

Example: `8 + 34; true ? 1 + 2 : 17;`

```
function eval_sequence(stmts)
{
  if (is_empty_sequence(stmts)) {
    return undefined;
  } else if (is_last_statement(stmts)) {
    return evaluate(first_statement(stmts));
  } else {
    const ignore =
      evaluate(first_statement(stmts));
    return eval_sequence(
      rest_statements(stmts)); }
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Frames and environments
Evaluating blocks
Evaluating declarations
Evaluating names

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations**
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation

Review: Adding blocks and declarations

stmt ::= ...
 | { *stmt* } block
 | *const name = expr* ; constant declaration
expr ::= ...
 | *name* name

Example:

```
const y = 4;  
{  
  const x = y + 7;  
  x * 2;  
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Frames and environments
Evaluating blocks
Evaluating declarations
Evaluating names

Evaluating the language

```
function evaluate(comp, env) {  
  return ...  
    : is_name(comp)  
    ? lookup_symbol_value(  
        symbol_of_name(comp), env)  
    : is_block(comp)  
    ? eval_block(comp, env)  
    : is_declaration(comp)  
    ? eval_declaration(comp, env)  
    : error(comp, "Unknown component:");  
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
 Adding blocks and declarations
 Adding simple compound functions
A small complication: return statements
 Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Frames and environments
Evaluating blocks
Evaluating declarations
Evaluating names

Review: Frames and environments

```
function make_frame(names, values) {  
    return pair(names, values);  
}  
  
const frame_names = head;  
const frame_values = tail;  
  
function extend_environment(ns, vs, e) {  
    return pair(make_frame(ns, vs), e);  
}  
  
const the_empty_environment = null;  
const the_global_environment  
    = the_empty_environment;
```

Evaluating blocks

Example of block statement:

```
{ const x = y + 7; x * 2; }
```

```
function eval_block(component, env) {  
  const body = block_body(component);  
  const locals = scan_out_declarations(body);  
  const unasgs = list_of_unasg(locals);  
  return evaluate(body,  
                  extend_environment(locals,  
                                    unasgs,  
                                    env));  
}  
  
const list_of_unasg =  
  symbols => map(symbol => "*unasg*", symbols);
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
 Adding blocks and declarations
 Adding simple compound functions
A small complication: return statements
 Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Frames and environments
Evaluating blocks
Evaluating declarations
Evaluating names

Review: Scanning out declarations

Example of block statement:

```
{ const x = y; { const y = 2;} const z = x/2; x;}

function scan_out_declarations(comp) {
    return is_sequence(comp)
        ? accumulate(
            append,
            null,
            map(scan_out_declarations,
                sequence_statements(comp)))
        : is_declaration(comp)
        ? list(declaration_symbol(comp))
        : null;
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
Evaluator
Review: Frames and environments
Evaluating blocks
Evaluating declarations
Evaluating names

Evaluating declarations

Example: `const x = y + 7;`

```
function eval_declaration(comp, env) {  
  assign_symbol_value(  
    declaration_symbol(comp),  
    evaluate(declaration_value_expression(comp),  
             env),  
    env);  
  return undefined;  
}
```

Review: Assign a name to a value

Example: `const x = 8 + 34;`

$\Rightarrow$ `assign_symbol_value("x", 42, env)`

```
function assign_symbol_value(symbol, val, env) {  
  function scan_write(symbols, vals) {  
    return symbol === head(symbols)  
      ? set_head(vals, val) // was: "*unasg*"   
      : scan_write(tail(symbols), tail(vals));  
  }  
  const frame = first_frame(env);  
  return scan_write(frame_symbols(frame),  
                    frame_values(frame));  
}
```

Review: Some more environment functions

```
function first_frame(env) {  
    return head(env);  
}
```

```
function enclosing_environment(env) {  
    return tail(env);  
}
```

```
function is_empty_environment(env) {  
    return is_null(env);  
}
```


On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

[Review: Syntax in BNF
Evaluator](#)
[Review: Frames and environments](#)
[Evaluating blocks](#)
[Evaluating declarations](#)
[Evaluating names](#)

Review: Looking up a symbol

```
function lookup_symbol_value(name, env)
{
  function env_loop(env) {
    function scan_read(names, vals) {...}
    if (is_empty_environment(env)) {
      error(name, "Unbound name:");
    } else {
      const frame = first_frame(env);
      const value = scan_read(frame_names(frame),
                              frame_values(frame));
      if (value === "*unasg*") {
        error(name, "Used before declared:");
      } else { return value;
    } } }
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
The evaluator
Review: Function objects
Review: Function declaration
Function application
Primitive functions

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions**
- 6 A small complication: return statements
- 7 Lazy Evaluation

Adding compound functions

```

stmt ::= ...
        | function name ( params )
          { return expr ; }    function declaration

expr ::= ...
        | expr( expr1, ..., exprn )    function application
        | params => expr                lambda expression

```

Example:

```

const fact = n =>
    n === 1 ? 1 : n * fact(n - 1);
fact(5); // result: 120

```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
The evaluator
Review: Function objects
Review: Function declaration
Function application
Primitive functions

Evaluating compound functions

```
function evaluate(comp, env) {  
  return ...  
  : is_lambda_expression(comp)  
  ? make_simple_fun(lambda_param_symbols(comp),  
                    return_expression(  
                      lambda_body(comp), env))  
  : is_function_declaration(comp)  
  ? evaluate(fun_decl_to_constant_decl(comp),  
            env)  
  : is_application(comp)  
  ? apply(evaluate(function_expression(comp), env),  
          list_of_values(arg_expressions(comp), env))  
  : error(comp, "Unknown component:"); }
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
The evaluator
Review: Function objects
Review: Function declaration
Function application
Primitive functions

Function objects

Example: $x \Rightarrow x + y$

```
function make_simple_fun(params, body, env) {  
    return list("simple_function",  
                params, body, env);  
}
```

Review: Transformation of function declaration

```
function fact(n) {  
    return n === 1 ? 1 : n * fact(n - 1);  
}  
  
// becomes  
const fact = n => n===1 ? 1 : n * fact(n - 1);  
  
function fun_decl_to_constant_decl(comp) {  
    return make_constant_declaration(  
        fun_declaration_name(comp),  
        make_lambda_expression(  
            fun_declaration_parameters(comp),  
            fun_declaration_body(comp)));  
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
The evaluator
Review: Function objects
Review: Function declaration
Function application
Primitive functions

Function application

Example: `fact(n - 1)`

```
function apply(fun, args) {  
  return is_primitive_function(fun)  
    ? apply_primitive_function(fun, args)  
    : is_simple_function(fun)  
    ? evaluate(function_body(fun),  
               extend_environment(  
                 function_parameters(fun),  
                 args,  
                 function_environment(fun))  
    : error(fun, "Unknown function type:");  
}
```

Declaring primitive functions

```
const primitive_functions =  
  list(pair("math_sin", math_sin,  
    ...  
    // operator combinations can be treated  
    // as primitive function applications  
    pair("+", (x, y) => x + y),  
    pair("-", (x, y) => x - y),  
    ...);  
function setup_environment () {  
  ...primitive_functions...  
}  
const the_global_environment =  
  setup_environment();
```


On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Review: Syntax in BNF
The evaluator
Review: Function objects
Review: Function declaration
Function application
Primitive functions

Review: Applying primitive functions

```
function apply_primitive_function(fun, arglist) {  
    return apply_in_underlying_javascript(  
        primitive_implementation(fun),  
        arglist);  
}
```

- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements**
- 7 Lazy Evaluation

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

The problems
The evaluator
Handling return statements
Return statements in sequences
Return values in apply

Return statements in JavaScript

The Problems

- Functions that do not evaluate a return statement must return undefined
- Evaluation of a return statement *anywhere* in the function body will return from the function with the result of evaluating the return expression

Review: Adding return statements

stmt ::= ...
 | return *expr* ; return statement

Example:

```
function f(x, error) {  
  if (error) {  
    return "error has occurred";  
  }  
  const result = something_complicated(x);  
  return result;  
}  
f(42, true);
```

Evaluating compound functions

```
function evaluate(comp, env) {  
  return ...  
  : is_return_statement(comp)  
  ? eval_return_statement(component, env)  
  : error(comp, "Unknown component:");  
}
```

Handling return statements

Example: `return n * fact(n - 1);`

```
function eval_return_statement(stmt, env) {  
    return make_return_value(  
        evaluate(return_statement_expression(stmt),  
            env));  
}  
  
function make_return_value(content) {  
    return list("return_value", content);  
}  
  
function is_return_value(value) {  
    return is_tagged_list(value, "return_value");  
}
```

Handling return values in sequences

```
function eval_sequence(stmts, env) {  
  if (is_empty_sequence(stmts)) {  
    return undefined;  
  } else if (is_last_statement(stmts)) {  
    return evaluate(first_statement(stmts), env);  
  } else {  
    const first_stmt_value =  
      evaluate(first_statement(stmts), env);  
    if (is_return_value(first_stmt_value)) {  
      return first_stmt_value;  
    } else {  
      return eval_sequence(  
        rest_statements(stmts), env);  
    }  
  }  
}
```

Handling return values in apply

```
function apply(fun, args) {  
  ...  
  const result = evaluate(...body...);  
  if (is_return_value(result)) {  
    return return_value_content(result);  
  } else {  
    return undefined;  
  } ...  
}
```


- 1 On metacircular evaluators
- 2 Calculator language
- 3 Adding booleans, conditionals, and sequences
- 4 Adding blocks and declarations
- 5 Adding simple compound functions
- 6 A small complication: return statements
- 7 Lazy Evaluation**

An example

```
unless(is_null(xs), head(xs), display("xs null"))
```

Can we implement unless like as follows?

```
function unless(condition, usual, exceptional) {  
    return condition ? exceptional : usual;  
}
```

Applicative order reduction, 1.1.5

```
function sq(x) { return x * x; }  
function sum_of_sqs(x,y) { return sq(x)+sq(y);}  
function f(a) { return sum_of_sqs(a+1, a*2);}
```

f(5)

-> sum_of_sqs(5 + 1, 5 * 2)

-> sum_of_sqs(6, 10)

-> sq(6) + sq(10)

-> (6 * 6) + (10 * 10)

⋮

-> 136

Normal order reduction (L2)

```
function sq(x) { return x * x; }  
function sum_of_sqs(x,y) { return sq(x)+sq(y);}   
function f(a) { return sum_of_sqs(a+1, a*2);} 
```

f(5)

-> sum_of_sqs(5 + 1, 5 * 2)

-> sq(5 + 1) + sq(5 * 2)

-> ((5 + 1) * (5 + 1)) +
((5 * 2) * (5 * 2))

⋮

-> 136

- On metacircular evaluators
 - Calculator language
 - Adding booleans, conditionals, and sequences
 - Adding blocks and declarations
 - Adding simple compound functions
 - A small complication: return statements
 - Lazy Evaluation**

Lazy evaluation

Laziness

In applications, arguments are not evaluated. They are passed to the function as expressions.

Force evaluation when needed

The argument expressions are evaluated when needed, during evaluation of body

Implementation

“Eager” evaluation (applicative order)

```
: is_application(comp)
? apply(evaluate(function_expression(comp),
               env),
        list_of_values(arg_expressions(comp),
                        env))
```

“lazy” evaluation (normal order)

```
: is_application(comp)
? apply(actual_value(function_expression(comp),
                     env),
        arg_expressions(comp),
        env)
```

Modifying apply

```
function apply(fun, args, env) {  
  if (is_primitive_function(fun)) {  
    return apply_primitive_function(  
      fun, list_of_arg_values(args, env));  
  } else if (is_compound_function(fun)) {  
    const result = evaluate(  
      function_body(fun),  
      extend_environment(  
        function_parameters(fun),  
        list_of_delayed_args(args,  
          env),  
        function_environment(fun)));  
    return is_return_value(result) ? ... : ...;}}}
```

Argument evaluation

```
function list_of_arg_values(exps, env) {  
  return map(exp => actual_value(exp, env), exps)  
}  
function list_of_delayed_args(exps, env) {  
  return map(exp => delay_it(exp, env), exps);  
}
```


Implementation of thunks

```
function delay_it(exp, env) {  
    return list("thunk", exp, env);  
}  
function is_thunk(obj) {  
    return is_tagged_list(obj, "thunk");  
}  
function thunk_exp(thunk) {  
    return head(tail(thunk));  
}  
function thunk_env(thunk) {  
    return head(tail(tail(thunk)));  
}
```

Implementation of forcing (simple version)

```
function force_it(obj) {  
    if (is_thunk(obj)) {  
        return actual_value(thunk_exp(obj),  
                             thunk_env(obj));  
    } else {  
        return obj;  
    }  
}
```

Implementation of forcing (with memoization)

```
function force_it(obj) {  
  if (is_thunk(obj)) {  
    const res = actual_value(thunk_exp(obj),  
                             thunk_env(obj));  
    set_head(obj, "evaluated_thunk");  
    set_head(tail(obj), res); // replace value  
    set_tail(tail(obj), null); // forget env  
    return res;  
  } else if (is_evaluated_thunk(obj)) {  
    return thunk_value(obj);  
  } else {  
    return obj;  
  }  
}
```

On metacircular evaluators
Calculator language
Adding booleans, conditionals, and sequences
Adding blocks and declarations
Adding simple compound functions
A small complication: return statements
Lazy Evaluation

Summary

- Metacircular evaluator:
the most beautiful program ever written (?)
- Similar to CSE machine (dispatch, environments)
- Control is implicit, using recursion
- evaluate—apply loop
- Lazy evaluation